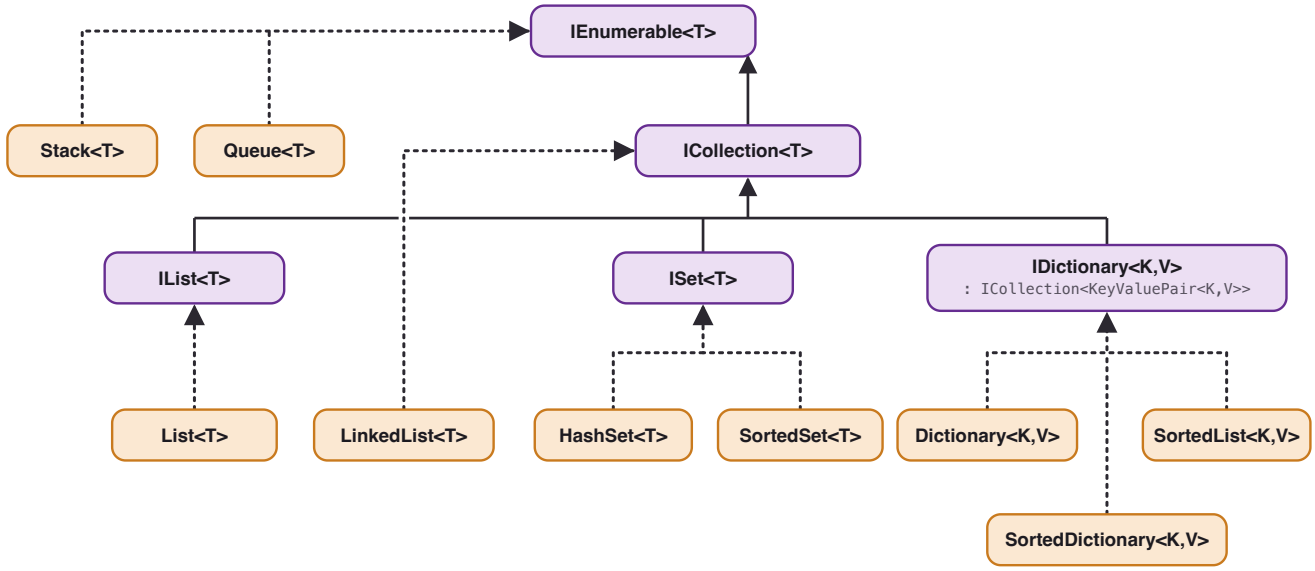
Colecțiile din .NET — Schema

Cine pe cine moștenește, în `System.Collections.Generic` · aceeași ierarhie în .NET Framework 4.x, la olimpiadă

Interfață · Clasă · → extinde (interfață din interfață) · ⇢ implementează

1. Cine pe cine moștenește



2. Ce colecție alegi

Întrebarea ta	Colecția	Ce folosești
„Am o înșiruire în ordine, cu poziții”	`List<T>`	`Add`, `Insert`, `RemoveAt`, `lista[i]`, `Sort`
„Adaug și scot des la capete, nu-mi trebuie poziții”	`LinkedList<T>`	`AddFirst`, `AddLast`, `RemoveFirst`, `RemoveLast`
„Caut ceva după o cheie (nume, cod)”	`Dictionary<K, V>`	`TryGetValue`, `ContainsKey`, `d[cheie] = valoare`
„Caut după cheie și vreau cheile în ordine”	`SortedDictionary<K, V>`	ca la `Dictionary`; `foreach` dă cheile crescător
„Cheile în ordine, dar și a i-a cheie”	`SortedList<K, V>`	ca la `Dictionary` + `Keys[i]`, `Values[i]`
„Am văzut deja asta?” / „fără dubluri”	`HashSet<T>`	`Add` (întoarce `false` dacă exista), `Contains`, `UnionWith`
„Fără dubluri și în ordine”	`SortedSet<T>`	`Add`, `Contains`, `Min`, `Max`; `foreach` crescător
„Procesez în ordinea sosirii”	`Queue<T>`	`Enqueue`, `Dequeue`, `Peek`
„Mă întorc la ultimul lucru”	`Stack<T>`	`Push`, `Pop`, `Peek`
„Știu sigur câte sunt, nu se schimbă”	tablou `T[]`	`t[i]`, `Length`, `Array.Sort`